

PEAK: PUF-Based End-to-End Authentication and Key-Exchange for IoT

Chandranshu Gupta, Gaurav Varshney, Gaurav Kumar, Satyadev Ahlawat

Abstract—We propose a lightweight end-to-end authentication and key-exchange protocol for resource-constrained IoT devices that avoids computationally expensive cryptographic operations by leveraging Physically Unclonable Functions (PUFs) and employs a lightweight Fuzzy Extractor (FE) to reliably handle PUF noise. The protocol relies exclusively on lightweight hash, XOR, and symmetric encryption/decryption operations. Moreover, the proposed protocol mitigates the storage overhead by avoiding the maintenance of large CRP repositories at the server. Security properties are formally verified using the scyther tool. A prototype implementation using two ESP32-based IoT nodes and a server further demonstrates its practicality for real-world deployments. Finally, benchmarking against state-of-the-art schemes indicates that the proposed protocol incurs comparatively low computation time and communication cost, highlighting its efficiency and lightweight nature.

Index Terms—IoT, Lightweight Authentication, PUF, Key-exchange, Fuzzy Extractor.

I. INTRODUCTION

The Internet of Things (IoT) interconnects a large number of heterogeneous devices for applications such as smart environments, healthcare, industrial automation, and Industrial IoT systems that increasingly exchange sensitive data, making secure authentication and session key establishment essential for system reliability and privacy [1]. Since IoT devices are often resource-constrained and deployed in unattended environments, they remain vulnerable to physical tampering, Man-in-the-Middle (MITM), replay, and impersonation attacks [2]. At the same time, expensive mechanisms such as PKI, IBE, and ECC are often impractical, while storing long-term secrets in non-volatile memory exposes devices to key-extraction attacks [3]. These challenges motivate lightweight security mechanisms for constrained IoT deployments. PUFs provide a hardware-rooted alternative by exploiting intrinsic manufacturing variations to generate unique Challenge-Response Pair (CRP) behavior without storing secrets [1].

Related Work: Existing PUF-based authentication and key-exchange schemes for IoT span PKI/certificate-based, pairing-based, ECC-based, and lightweight symmetric designs [1]–[5]. However, many still incur substantial computation and communication overhead due to public-key operations, require large CRP repositories at the verifier [6], or insufficiently address noisy PUF responses through explicit error correction

[1], [5]. Even lightweight symmetric PUF-based protocols often suffer from trade-offs such as higher MAC/communication overhead, lack of mutual authentication, or vulnerability to common active attacks [1], [5]. These limitations motivate a lightweight, experimentally validated end-to-end PUF-based protocol that simultaneously reduces computation and communication overhead while improving scalability, security, and robustness to PUF noise.

Contributions: This letter presents a lightweight PUF-based end-to-end mutual authentication and key-exchange protocol for resource-constrained IoT devices that avoids permanent secret storage and expensive public-key operations. The design uses role-separated PUF-derived keys, where one key supports authentication and session-key derivation, while the other supports secure encrypted state update. It further avoids large verifier-side CRP repositories by maintaining only a constant-size active challenge-key state per device, while a lightweight FE enables reliable operation under noisy PUF responses. Formal verification using Scyther and experimental validation on both SRAM and Arbiter PUF platforms demonstrate the practicality, security, and efficiency of the proposed scheme.

II. PROPOSED PROTOCOL

The network model consists of two resource-constrained IoT devices, denoted as IoT_A and IoT_B , and a trusted server S , communicating over an insecure public channel vulnerable to common attacks such as eavesdropping and replay. We assume a trusted and uncompromised server S and an active Dolev–Yao adversary on the public channel. A device may be physically captured and its stored helper data may be read or tampered with; however, long-term secret keys are not stored in non-volatile memory, since (K_{X1}, K_{X2}) are reconstructed on demand from PUF responses using the fuzzy extractor. Helper data leakage alone is insufficient to recover the secrets, while tampering it only causes reconstruction failure and protocol abort rather than successful impersonation. The proposed protocol operates in two phases: an enrollment phase and an authentication and key-exchange phase.

A. Enrollment Phase

During the enrollment phase, the trusted server S interacts with each IoT device $X \in \{A, B\}$ in a secure environment. For each device, S provides two PUF challenges (C_{X1}, C_{X2}) , which are applied to the device PUF to obtain noisy responses (R_{X1}, R_{X2}) . Each response is processed locally using a fuzzy extractor to derive stable keys (K_{X1}, K_{X2}) along with corresponding helper data (HD_{X1}, HD_{X2}) . The helper data is stored internally at the device, while the server stores only the associated challenges and derived keys from raw noisy PUF

Manuscript received xxx; revised xxx.

Chandranshu Gupta and Gaurav Varshney are with Department of Computer Science and Engineering, Indian Institute of Technology Jammu, India. (e-mail: {chandranshu.gupta, gaurav.varshney}@iitjammu.ac.in) (Corresponding Author: Chandranshu Gupta)

Gaurav Kumar and Satyadev Ahlawat are with Department of Electrical Engineering, Indian Institute of Technology Jammu, India. (e-mail: {gaurav.kumar, satyadev.ahlawat}@iitjammu.ac.in)

responses. Thus, the server maintains only a constant-size per-device state, namely the currently active challenge-key entries, rather than a large CRP repository. In each session, these entries are refreshed through encrypted update messages that replace the old values, thereby reducing server-side storage and management overhead. This one-time enrollment enables reliable key reconstruction during authentication despite PUF noise.

B. Authentication and Key Exchange

The authentication and key-exchange protocol is illustrated in Fig. 1 and proceeds as follows:

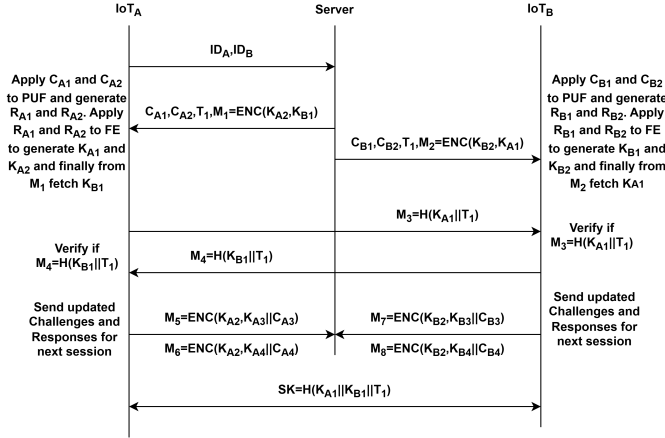


Fig. 1. End-to-End Authentication and Key-Exchange Protocol

- 1) **Session Initiation:** Device *A* initiates the protocol by sending the identities of both devices to the server:

$$A \rightarrow S : ID_A, ID_B$$

- 2) **Challenge Distribution:** The server selects stored challenges and a freshness token T_1 , and transmits them along with encrypted key material corresponding to each device:

$$S \rightarrow A : C_{A1}, C_{A2}, T_1, M_1 = \text{ENC}(K_{A2}, K_{B1})$$

$$S \rightarrow B : C_{B1}, C_{B2}, T_1, M_2 = \text{ENC}(K_{B2}, K_{A1})$$

- 3) **PUF Evaluation and Key Reconstruction:** Each device applies the received challenges to its PUF and uses the fuzzy extractor with stored helper data to regenerate stable keys:

$$(R_{A1}, R_{A2}) \xrightarrow{\text{FE}} (K_{A1}, K_{A2})$$

$$(R_{B1}, R_{B2}) \xrightarrow{\text{FE}} (K_{B1}, K_{B2})$$

The encrypted messages are then decrypted to obtain the peer device key. Here, encryption is not driven by pre-stored long-term keys; instead, the wrapping keys (K_{A2}, K_{B2}) are reconstructed on demand from PUF responses, while (K_{A1}, K_{B1}) are reserved for authentication and session-key derivation.

- 4) **Mutual Authentication:** Device *A* and *B* confirm possession of correct keys using hash-based authenticators:

$$A \rightarrow B : M_3 = H(K_{A1} || T_1)$$

$$B \rightarrow A : M_4 = H(K_{B1} || T_1)$$

Each device verifies the received hash; a mismatch results in immediate termination.

Scyther results : verify				
Claim			Status	Comments
PUFAKEWithServerAccept	S	PUFAKEWithServerAccept_S1	Secret PUF(A,C_A2)	OK Verified No attacks.
		PUFAKEWithServerAccept_S2	Secret PUF(B,C_B2)	OK Verified No attacks.
		PUFAKEWithServerAccept_S3	Secret H(SKtag,PUF(B,C_B1),PUF(A,C_A1),T1)	OK No attacks within bounds.
		PUFAKEWithServerAccept_S4	Alive	OK Verified No attacks.
		PUFAKEWithServerAccept_S5	Weakagree	OK No attacks within bounds.
		PUFAKEWithServerAccept_S6	Niagree	OK No attacks within bounds.
		PUFAKEWithServerAccept_S7	Nisynch	OK No attacks within bounds.
	A	PUFAKEWithServerAccept_A1	Secret H(SKtag,PUF(B,C_B1),PUF(A,C_A1),T1)	OK No attacks within bounds.
		PUFAKEWithServerAccept_A2	Secret PUF(A,C_A2)	OK No attacks within bounds.
		PUFAKEWithServerAccept_A3	Alive	OK No attacks within bounds.
		PUFAKEWithServerAccept_A4	Weakagree	OK No attacks within bounds.
		PUFAKEWithServerAccept_A5	Niagree	OK No attacks within bounds.
		PUFAKEWithServerAccept_A6	Nisynch	OK No attacks within bounds.
	B	PUFAKEWithServerAccept_B1	Secret H(SKtag,PUF(B,C_B1),PUF(A,C_A1),T1)	OK No attacks within bounds.
		PUFAKEWithServerAccept_B2	Secret PUF(B,C_B2)	OK No attacks within bounds.
		PUFAKEWithServerAccept_B3	Alive	OK No attacks within bounds.
		PUFAKEWithServerAccept_B4	Weakagree	OK No attacks within bounds.
		PUFAKEWithServerAccept_B5	Niagree	OK No attacks within bounds.
		PUFAKEWithServerAccept_B6	Nisynch	OK No attacks within bounds.
Done.				

Fig. 2. Scyther analysis results

- 5) **CRP Update for Forward Security:** To support future sessions, updated challenge-key pairs are securely transmitted to the server:

$$A \rightarrow S : M_5 = \text{ENC}(K_{A2}, K_{A3} || C_{A3}),$$

$$M_6 = \text{ENC}(K_{A2}, K_{A4} || C_{A4})$$

$$B \rightarrow S : M_7 = \text{ENC}(K_{B2}, K_{B3} || C_{B3}),$$

$$M_8 = \text{ENC}(K_{B2}, K_{B4} || C_{B4})$$

The server updates its challenge database with these new CRPs. Here, the next challenges are generated from the currently active challenge state, and the freshness token by performing XOR operation.

- 6) **Session Key Derivation:** Upon successful authentication, both devices independently compute the session key:

$$SK = H(K_{A1} || K_{B1} || T_1)$$

III. SECURITY ANALYSIS OF THE PROPOSED PROTOCOL

This section presents both formal and informal analyses to demonstrate the security of the proposed protocol.

A. Formal Security Automation Testing via Scyther Tool

We formally analyzed the proposed protocol using *Scyther*, a symbolic verification tool for security protocols specified in SPDL. The analysis assumes an active Dolev-Yao adversary controlling the communication channel, while cryptographic primitives are modeled as ideal. The protocol was encoded in SPDL with its roles, message flow, PUF-derived values, and encrypted payloads, and the security objectives were expressed using the Alive, Weakagree, Niagree, and Nisynch claims. Verification was performed using *Scyther-Linux* v1.2.1. The results show that the proposed protocol satisfies the stated secrecy and agreement-related claims for the tested scenarios. A summary is given in Fig. 2.

B. Informal Security Analysis

Mutual Authentication: Mutual authentication is achieved via proof-of-possession of the relayed first responses, bound to the freshness token T . Device *B* sends: $B \rightarrow A : M_4 = H(K_{B1} || T)$ and device *A* verifies it by recomputing $H(K_{B1} || T)$ using K_{B1} recovered from M_1 and T received from S . Symmetrically, device *A* sends: $A \rightarrow B : M_3 = H(K_{A1} || T)$, and device *B* verifies it using K_{A1} recovered from M_2 and the same

T . An external adversary cannot compute M_3 without knowledge of K_{A1} , nor compute M_4 without knowledge of K_{B1} . Since K_{B1} and K_{A1} are delivered to A and B respectively only inside $\text{AES}_{K_{A2}}(\cdot)$ and $\text{AES}_{K_{B2}}(\cdot)$, the attacker cannot obtain these values without breaking symmetric encryption. Therefore, successful verification of M_3 and M_4 implies that the peer device possesses the corresponding secret response, providing mutual authentication.

Session Key Derivation and Secrecy: After successful verification in the mutual authentication step, both devices derive a common session key. In the protocol, the session key is computed as: $SK = H(K_{A1} \parallel K_{B1} \parallel T)$. The adversary observes at most hash outputs and ciphertexts, but does not learn K_{B1} or K_{A1} . Under the one-wayness of $H(\cdot)$ and confidentiality of AES under uncompromised keys, the adversary cannot compute SK . Hence, SK remains secret to A and B .

Replay Resistance and Freshness Enforcement: The protocol binds the authentication tags M_3 and M_4 to the freshness token T . Consequently, replaying an old hash value from a previous session (computed with T') will fail verification under the current session token T . Since K_{A2} and K_{B2} are never transmitted in the clear, an external attacker cannot decrypt M_5 – M_8 to learn the newly generated PUF responses. The server verifies that the received updates are consistent with the prescribed XOR challenge derivations, and then replaces the stored CRPs with the new values. This *stateful CRP evolution* ensures that subsequent sessions rely on fresh challenges and responses, and stale CRPs cannot be reused.

Resistance to Impersonation and MITM Attacks: An attacker attempting to impersonate B to A must forge $M_4 = H(K_{B1} \parallel T)$ without learning K_{B1} . Because K_{B1} is only available to A after decrypting M_1 under K_{A2} , the attacker cannot construct M_4 unless it breaks AES or compromises A or S . The same argument applies for impersonation of A to B . Moreover, a man-in-the-middle adversary cannot cause A and B to accept different session keys, since both keys are deterministically derived from the same pair (K_{A1}, K_{B1}) and bound to T . Any modification to transmitted values would fail decryption or hash verification.

TABLE I
AVERAGE EXECUTION TIME OF THE CRYPTOGRAPHIC PROTOCOLS

Operation	Average time (microseconds)
T_{ECPM}	161090.45
T_{Hash}	31.9
T_{AES}	43.9
T_{MAC}	137.45
T_{XOR}	1.65
T_{PUF}	5.4
T_{FE}	18.9

IV. EXPERIMENTS AND PERFORMANCE EVALUATION

We evaluated the proposed protocol using two PUF implementations: SRAM PUF on ESP32 and Arbiter PUF (APUF) on a Xilinx Pynq-Z2 FPGA. SRAM-based PUF responses were extracted from ESP32 modules by capturing their SRAM power-up values across multiple power-cycle measurements from a 2KB memory region, yielding 8-bit responses per address. Reliability and uniqueness were measured using intra-device and inter-device Hamming Distance (HD), respectively,

giving 96% reliability and 49.8% uniqueness. Our APUF implemented on Xilinx Pynq z2 FPGA board, comprising a 64-stage delay circuit, each stage built using LUT-based multiplexers. A D flip-flop initiates simultaneous transitions across symmetric delay paths, and a NAND-based latch acts as the arbiter to determine the faster path. APUF achieved an intra-device reliability of 98.9% and an inter-device uniqueness of 49.7%. Although classical APUFs can be vulnerable to machine learning/modeling attacks under large-scale CRP exposure, the proposed protocol restricts public CRP exposure by provisioning only two enrolled challenges per device and by carrying subsequent CRP updates in encrypted form rather than transmitting raw PUF responses over the public channel. Thus, the protocol reduces the observable data typically required for conventional modeling attacks. To address noise in PUF outputs, a lightweight FE based on a syndrome-based BCH(15, 5, 3) code was employed. A 128-bit response is zero-padded to 135 bits and partitioned into nine 15-bit blocks; each block yields 10 bits of helper data, resulting in 90 bits of helper data in total, and the corrected blocks are recombined to reconstruct the stable response. On the FPGA, the BCH encoder occupies 14 LUTs, 44 flip-flops, and 15 slices, while the decoder occupies 113 LUTs, 49 flip-flops, and 36 slices, confirming modest hardware overhead.

The computational time of individual cryptographic operations was evaluated using an ESP32-WROOM platform with a 240 MHz dual-core Xtensa LX6 processor, 520 KB SRAM, and 4 MB flash, using Arduino IDE with ESP32 and mbedTLS. Cryptographic primitives (SHA-256, AES-128-GCM, HMAC-SHA256, and secp256r1 ECC) were benchmarked via mbedTLS by averaging 1000 runs (100 for ECC), as summarized in Table I. APUF and FE operations were implemented on a Xilinx Pynq-Z2 FPGA. Our proof of concept protocol was implemented on two ESP32-WROOM modules as IoTA and IoTB, and on a desktop PC (Intel i7-9700K, 16 GB RAM, Ubuntu 20.04) running Python as the trusted Server. We instrumented microsecond timers on the ESP32 and Server to measure computation time for each protocol primitive.

For fairness, we normalize each scheme over one complete secure-session cycle, including authentication, key establishment, and subsequent session-dependent update/refresh operations whenever fresh CRP/state evolution is required for continued secure operation. One-time enrollment and offline provisioning are excluded. Table II compares the computational overhead of the proposed protocol with representative PUF-based authentication schemes from the literature. The total cost is computed from the number of primitive invocations executed by each entity over the complete protocol exchange. To ensure a fair and normalized comparison, we assume that a FE is incorporated in all schemes, as PUF noise handling is essential in practical deployments. As shown in Table II, the proposed protocol avoids expensive elliptic curve point multiplication operations, which constitute the dominant computational cost in several existing schemes [2], [3], [7], thereby resulting in significantly lower overall computation time. Compared with recent ECC-free schemes [8]–[10], the proposed protocol also achieves lower total computation cost due to a more compact use of lightweight operations. The

TABLE II
QUANTITATIVE COMPARISON OF COMPUTATION OVERHEAD

Ops	[5]			[1]			[2]			[7]			[3]			[8]			[9]			[10]			Our Work		
	A	S	B	A	S	B	A	S	B	A	S	B	A	S	B	A	S	B	A	S	B	A	S	B	A	S	B
N_{xor}	-	-	-	7	7	7	-	-	-	2	-	2	2	-	2	-	-	1	-	1	3	7	4	2	2	4	2
N_{puf}	2	-	2	2	-	2	1	-	1	1	2	1	2	-	2	2	2	2	-	2	2	-	2	4	-	4	
N_{fe}	2	-	2	2	-	2	1	-	1	1	2	1	2	-	2	2	2	1	2	1	2	-	2	4	-	4	
N_h	1	2	1	3	4	3	2	-	2	3	6	3	5	6	5	5	8	5	10	7	5	12	25	13	3	-	3
N_{mac}	4	4	4	-	-	-	1	2	1	-	-	-	-	-	-	3	2	3	-	-	-	-	-	-	-	-	-
N_{ed}	3	4	3	-	-	-	2	-	2	1	2	1	3	6	3	3	2	3	3	3	2	-	-	-	3	6	3
N_{ecm}	-	-	-	-	-	-	5	2	4	1	-	1	2	-	2	-	-	-	-	-	-	-	-	-	-	-	-
T_{sum} (ms)	2.3132			0.451			1772.897			322.837			645.496			2.160			1.154			1.715			0.926		
N_{xor} : XOR; N_{puf} : PUF; N_{fe} : Fuzzy extractor; N_h : Hash; N_{mac} : HMAC; N_{ed} : encrypt/decrypt; N_{ecm} : EC point mult.; T_{sum} : Total time; A: IoT _A , S: Server, B: IoT _B																											

N_{xor} : XOR; N_{puf} : PUF; N_{fe} : Fuzzy extractor; N_h : Hash; N_{mac} : HMAC; N_{ed} : encrypt/decrypt; N_{ecm} : EC point mult.; T_{sum} : Total time; A: IoT_A, S: Server, B: IoT_B

scheme in [1] achieves lower computation time; however, it does not provide mutual authentication and is vulnerable to MITM and impersonation attacks. In contrast, the proposed protocol achieves comprehensive security guarantees while maintaining low computational overhead, offering a favorable trade-off between efficiency and security.

TABLE III
COMPARISON OF COMMUNICATION COST IN BITS

Schemes	IoT _A	IoT _B	Server	Total
[5]	3264	1920	2688	7872
[1]	800	832	3584	5216
[2]	1664	160	2800	4624
[7]	1088	1024	3104	5216
[3]	2304	2304	2560	7168
[8]	2112	1536	3712	7360
[9]	3330	1024	2304	6658
[10]	1312	3904	3072	8288
Our Work	1344	1280	2048	4672

The communication cost is obtained by summing the sizes of all transmitted fields in the authentication and key-exchange messages reported for each scheme; thus, the comparison is message-structure based and independent of any specific network latency or channel condition model. For communication overhead analysis, we assume 256-bit representations for random numbers, timestamps, hash and MAC outputs, private keys, FE outputs, PUF challenges, and responses, while device identifiers are assumed to be 32 bits in length. Elliptic-curve points are assumed to be 512 bits when transmitted in uncompressed form. For schemes where the original protocol or message structure implies compressed public-key representation, a 256-bit ECC/public-key size is used accordingly. For protocol-specific fields not covered by these normalized primitives, such as session identifiers, wrapped-key fields, or certificate-related fields, we used the bit lengths explicitly stated or directly implied in the original papers. Table III summarizes the communication cost comparison. As shown, the proposed protocol incurs lower total communication overhead than the compared schemes, including recent ECC-free protocols [8]–[10], while simultaneously supporting mutual authentication and resistance to common active attacks. These results demonstrate the communication efficiency of the proposed protocol for resource-constrained IoT deployments.

V. CONCLUSION

This work presented a lightweight PUF-based authentication and key-exchange protocol tailored for resource-constrained IoT devices that avoids permanent secret storage and relies on lightweight cryptographic primitives. By integrating a lightweight FE, the proposed scheme reliably handles noisy

PUF responses while eliminating the need to store or transmit raw PUF data and avoiding the maintenance of large CRP repositories at the server. The protocol achieves mutual authentication, fresh session key establishment, and forward secrecy with low computational, and communication overhead. Formal security analysis and experimental validation confirm resistance against common attacks such as MITM, replay, and impersonation attacks. Implementation results on ESP32 and FPGA-based platforms demonstrate the practicality and efficiency of the proposed scheme for real-world IoT deployments. Future work will investigate more lightweight error-correction techniques for PUF stabilization and evaluate the protocol under diverse environmental conditions.

REFERENCES

- [1] S. Roy, D. Das, A. Mondal, M. H. Mahalat, B. Sen, and B. Sikdar, "PLAKE: PUF-based secure lightweight authentication and key exchange protocol for IoT," *IEEE Internet of Things Journal*, vol. 10, no. 10, pp. 8547–8559, 2022.
- [2] S. Tran, C. T. Ngo, J.-P. Hong, et al., "A lightweight ECC-compatible end-to-end security protocol using CRP-PUF and TRNG for IoT devices," *IEEE Internet of Things Journal*, 2025.
- [3] S. Manivannan, R. S. Chakraborty, I. Chakraborti, and J. Ramalingam, "Practical and efficient PUF-based protocol for authentication and key agreement in IoT," *IEEE Embedded Systems Letters*, vol. 16, no. 2, pp. 118–121, 2023.
- [4] S. Li, T. Zhang, B. Yu, and K. He, "A provably secure and practical PUF-based end-to-end mutual authentication and key exchange protocol for IoT," *IEEE Sensors Journal*, vol. 21, no. 4, pp. 5487–5501, 2020.
- [5] M. N. Aman, K. C. Chua, and B. Sikdar, "Mutual authentication in IoT systems using physical unclonable functions," *IEEE Internet of Things Journal*, vol. 4, no. 5, pp. 1327–1340, 2017.
- [6] U. Chatterjee, V. Govindan, R. Sadhukhan, D. Mukhopadhyay, R. S. Chakraborty, D. Mahata, and M. M. Prabhu, "Building puf based authentication and key exchange protocol for iot without explicit crps in verifier database," *IEEE transactions on dependable and secure computing*, vol. 16, no. 3, pp. 424–437, 2018.
- [7] S. Roy, M. H. Mahalat, and B. Sen, "Design and Implementation of Cost-Effective End-to-End Authentication Protocol for PUF-Enabled IoT Devices," *IEEE Transactions on Emerging Topics in Computing*, 2025.
- [8] Y. Zheng, W. Liu, C. Gu, and C.-H. Chang, "Puf-based mutual authentication and key exchange protocol for peer-to-peer iot applications," *IEEE Transactions on Dependable and Secure Computing*, vol. 20, no. 4, pp. 3299–3316, 2022.
- [9] F. Zerrouki, S. Ouchani, and H. Bouarfa, "T2s-makep and t2t-makep: A puf-based mutual authentication and key exchange protocol for iot devices," *Internet of Things*, vol. 24, p. 100953, 2023.
- [10] A. M. A. Modarres, N. S. Anzabi-Nezhad, and M. Zare, "A new puf-based protocol for mutual authentication and key agreement between three layers of entities in cloud-based iomt networks," *IEEE Access*, vol. 12, pp. 21807–21824, 2024.